Ders 4 Genişletilmiş Notu: Shell Sort Ayrıntıları ve Sıralama Algoritması Seçimi

Ders 4 Genişletilmiş Notu: Shell Sort Ayrıntıları ve Sıralama Algoritması Seçimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. n^2 Karmaşıklıklı Algoritmaları Karşılaştırmak
- 2. Insertion Sort Neden Selection Sort'tan Daha Avantajlı?
- 3. Shell Sort'un Fikri
- 4. Adım Adım Shell Sort Örneği
- 5. Örnek Üzerine Öğrenci Soruları
- 6. Shell Sort'un Akış Diyagramı
- 7. Sıralama Algoritması Seçimi Üzerine Sorular
- 8. Exchange Yerine Kaydırma: `tmp` ile İyileştirme
- 9. Shell Sort Ne Zaman Kullanılmalı?
- 10. Çalışma Süresini Deneysel Olarak Ölçmek
- 11. Akış ve Kod Üzerine Son Sorular

1. n^2 Karmaşıklıklı Algoritmaları Karşılaştırmak

Hoca derse, bir önceki derste konuşulan sıralama algoritmalarını hatırlatarak başladı: Bubble Sort, Selection Sort, Insertion Sort ve en son Shell Sort. Bunlar birinci sınıfta, giriş derslerinde görülen algoritmalardır. İlk üçü en kötü durumda (worst case) n^2 karmaşıklıkla çalışır. O zaman doğal bir soru çıkar: Hepsi n^2 ise hangisini kullandığımız fark eder mi?

Hoca tahtada en kötü ve en iyi durumları yan yana yazdı, ortalama durumu da (average case) sözlü ekledi:

Algoritma	En kötü durum	Ortalama durum	En iyi durum
Bubble Sort	n^2	n^2	n
Selection Sort	n^2	n^2	n^2
Insertion Sort	n^2	n^2	n

En iyi durumlar üzerinde çok durulmaz, çünkü en iyi durum çok nadir karşılaşılan bir durumdur. Selection Sort için zaten çare yoktur, her zaman n^2 olur. Genelde karşılaşılan durum ortalama durumdur: Dizi ne küçükten büyüğe ne büyükten küçüğe sıralıdır, istatistiksel olarak ortalama bir karışıklıktadır. Ortalama durumda da üç algoritmanın karmaşıklığı n^2'dir.

> **Ders sonrası not:** Bubble Sort'un en iyi durumda n olması, yalnız bir turda hiç yer değiştirme olmazsa duran (`swap` değişkenli) sürüm için geçerlidir. Bayraksız sürüm sıralı dizide de bütün turları döner ve $n(n-1)/2$ karşılaştırma yapar.

1.1 Bubble Sort Neden Bu Kadar Kötülenir?

Hoca, herkesin Bubble Sort'u en düşük performanslı algoritma olarak bildiğini söyleyip sebebini sordu. Bir öğrenci "sürekli yer değiştirme yapıyor" dedi ve hoca bunu doğruladı.

Sorun yer değiştirmenin kendisi değildir. Sıralamak için bütün algoritmalar eninde sonunda yer değiştirir. Fark şurada: Diğer algoritmalar önce bir takım kontroller yapar, sonra bir noktada "tamam, bunun yerini değiştirelim" der. Bubble Sort ise bütün elemanlara ikişer ikişer, arka arkaya bakar ve her kontrolün sonunda yer değiştirme ihtiyacı doğabilir. Yer değiştirme de bir değiş tokuş (exchange) işlemidir: A'daki B'nin gözüne, B'deki A'nın gözüne yazılır. Bu yüzden daha masraflıdır.

> **Hocanın vurgusu:** Bu dönem artık "iç içe iki döngüyle yazdım, çalışıyor, o hâlde iyidir" düzeyinin ötesine geçiyoruz. Yazdığımız bir algoritmanın verimliliği için nelere dikkat etmemiz ve onu nasıl iyileştirebileceğimiz de gündemimizde.

2. Insertion Sort Neden Selection Sort'tan Daha Avantajlı?

Bu gözle bakıldığında, üç n^2 algoritma içinde en verimlisi Insertion Sort'tur. Çünkü Insertion Sort, baş tarafın kendi içinde sıralı olması avantajından faydalanır.

Hocanın örneği: Baş tarafta `3 5 7` yerleşmiş olsun, sırada 6 olsun.

- 6, 7'den küçük mü? Evet. O zaman 7 bir sağa gelir.
- 6, 5'ten küçük mü? Hayır. 6, 5'ten küçük değilse 3'ten de kesinlikle küçük değildir. 3'e bakmaya hiç gerek yoktur.

Sonuç `3 5 6 7` olur ve yalnız 2 karşılaştırma yapılır. Başa doğru gidilmez, geri kalan karşılaştırmalar hiç yapılmaz.

Hoca burada bir soru sordu: Selection Sort da böyle yapmıyor mu? O da dizinin en küçük elemanını bulup başa koyuyor, yerleştirdiği kısma bir daha bakmıyor. O hâlde Insertion Sort neden hâlâ daha avantajlı?

Soru: Selection Sort yerleştirdiği kısma bir daha bakmıyorsa, Insertion Sort neden daha avantajlı?

Cevap: Bir öğrenci önce tersini savundu: Selection Sort'ta en küçük bulunduğu için başa bakmaya gerek kalmıyor, Insertion Sort'ta ise başa kadar gitmek gerekebiliyor. Hoca "Ama Insertion'ı savunman lazım" diye uyardı. Başka bir öğrenci doğru cevabı verdi: Selection Sort'ta kalan elemanların hepsi arasında en küçüğü bulmak gerekir. Insertion Sort'ta ise hangi elemandan başlanacağı bellidir ve bir adım sonra, hatta hemen soldaki elemana bakınca bile durulabilir. Hoca bunu onayladı: Insertion Sort, bütün elemanlar içinde en küçüğü bulup yerleştirme derdinde değildir. Yalnız sırası gelen elemanın baş tarafa göre yerini belirler, yerini bulduğu noktada da durur.

Hoca, farkın çok büyük olmadığını ama Insertion Sort'un birazcık daha avantajlı olmasının ve n^2 algoritmalar içinde tercih edilmesinin sebebinin bu olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Fark sayıyla da görülebilir. Selection Sort her durumda $n(n-1)/2$ karşılaştırma yapar. Insertion Sort, elemanları farklı rastgele bir dizide ortalama yaklaşık $n^2/4$ karşılaştırma yapar, yani yaklaşık yarısını. Mertebe yine n^2 olduğu için hocanın "çok fark etmiyor" sözü karmaşıklık açısından doğrudur.
>
> **Hocanın vurgusu:** Insertion Sort'un en avantajlı olduğu durum **yarı sıralı** dizilerdir.

Yarı sıralı dizi, zamanında sıralı olup bazı işlemlerden sonra sırası bozulmuş dizidir. Bir kısmı zaten sıralı durduğu için sıralarken çok fazla eleman hareketi olmaz.

3. Shell Sort'un Fikri

Shell Sort, Insertion Sort mantığının üzerine kurulu bir algoritmadır. Hangi noktayı iyileştirmeye çalıştığını görmek için hoca tersten sıralı bir dizi düşünmemizi istedi. Baş taraf kendi

içinde sıralanmış, sırada küçük bir eleman, örneğin 2 var. Dizi tersten sıralıysa küçük elemanlar hep sonlarda kalmıştır. Onları başa taşımak için aradaki bütün elemanları tek tek kontrol ederek geçmek gerekir. Hoca bunun bir dezavantaj olarak değil, "daha nasıl iyileştiririz?" diye bakılan bir nokta olarak görülmesini istedi.

Shell Sort'un cevabı şudur: Birer birer karşılaştırmak yerine **zıplaya zıplaya** gidelim. Sondaki bir elemanı kendinden öncekilerin hepsiyle tek tek karşılaştırmak yerine, örneğin 5 öncesiyle karşılaştırırız. Ondan küçükse onun da 5 öncesiyle karşılaştırırız. Böylece küçük eleman başa çok daha hızlı taşınır.

3.1 Shell Sort'un Karmaşıklığı

Hoca, matematiksel doğrulamasını yapmayacağını belirterek en kötü durum için şunları söyledi: Çok iyi bir adım aralığı seçilirse (dörder dörder mi, beşer beşer mi ilerleneceği) en kötü durumda karmaşıklık $n^{3/2}$ civarında olur. Üs 1,5'tir, yani 2'den küçüktür. Seçilen adım büyüklüğüne bağlı olarak $n^{4/3}$ ve $n^{5/4}$ gibi değerler de görülür. Yani adım büyüklüğüne bağlı olarak genellikle n^2'den birazcık daha iyidir.

Bunun neden önemli olduğunu göstermek için hoca, n^2, n ve $n \log n$ sürelerinin birbirine göre durumunun görüldüğü tabloya geri döndü. n büyüdükçe, 1 milyon, 2 milyon oldukça, n^2 ile $n^{3/2}$ arasında da performans farkı oluşur. Bu yüzden insanların bunlarla uğraşması boşuna değildir.

> **Ders sonrası not:** Bu üsler farklı adım dizilerine aittir. Derste kullanılan $h = N/3$ dizisi için bir üs söylenmedi. $n^{5/4}$ değeri büyük olasılıkla kanıtlanmış bir üst sınır değil, deneysel bir ortalama durum ölçümüdür.

4. Adım Adım Shell Sort Örneği

Hoca, algoritma tasarımına geçmeden önce mantığı 12 elemanlı bir dizi üzerinde gösterdi:

```
indeks:  0   1   2   3   4   5   6   7   8   9  10  11
dizi   : 21   8  13  19  16  11   7  25  12  10   5   4
```

4.1 Adım Büyüklüğünün Seçimi

Bir elemanın kaç geriye taşınacağına bakmak için bir adım büyüklüğü (step size) seçilir. Adım büyüklüğünün nasıl seçileceğine dair farklı yöntemler ve çalışmalar vardır. $N/2$ alanlar da vardır, başka adım büyüklükleri kullananlar da. Hoca, genel kullanılan yöntemlerden biri olarak $h = N/3$ ile gösterdi. Burada $N = 12$ olduğu için $h = 4$'tür.

$h = 4$ olması, birbirinden 4 uzaktaki gözlerin kendi içinde değerlendirileceği anlamına gelir. Hoca bu grupları tahtada farklı renklere boyadı ve her elemanın yalnız kendi rengindekilerle karşılaştırıldığını özellikle gösterdi:

Alt dizi (gözler)	Başlangıçtaki değerler	Tur sonundaki değerler
0, 4, 8	21, 16, 12	12, 16, 21
1, 5, 9	8, 11, 10	8, 10, 11
2, 6, 10	13, 7, 5	5, 7, 13
3, 7, 11	19, 25, 4	4, 19, 25

> **Ders sonrası not:** Hoca $h = 4$ için "3 tane parçam var" dedi; doğrusu 4 alt dizi vardır ve her alt dizide 3 eleman bulunur.

4.2 $h = 4$ Turu

İşe h. gözden, yani 4. gözden başlanır. Her eleman 4 gerisindekiyle karşılaştırılır. Küçükse yer değiştirir ve 4 geri gidilerek devam edilir. 4 gerisi olmayan bir göze gelindiğinde durulur. Hoca hem exchange hem de Insertion Sort'taki gibi kaydırma yapılabileceğini, ikisinin de mümkün olduğunu söyledi. Aşağıdaki tabloda her i turunun sonundaki dizi yer alır.

i	Eleman	Karşılaştırıldığı değerler	Ne oldu?	Tur sonunda dizi
4	16	21	16 küçük, 0. göze geçti. 0'ın 4 gerisi yok, durdu.	`16 8 13 19 21 11 7 25 12 10 5 4`
5	11	8	11 büyük, değişmedi.	`16 8 13 19 21 11 7 25 12 10 5 4`
6	7	13	7 küçük, 2. göze geçti. 2'nin 4 gerisi yok, durdu.	`16 8 7 19 21 11 13 25 12 10 5 4`
7	25	19	25 büyük, değişmedi.	`16 8 7 19 21 11 13 25 12 10 5 4`
8	12	21, 16	İkisinden de küçük, 0. göze geldi.	`12 8 7 19 16 11 13 25 21 10 5 4`
9	10	11, 8	11'den küçük, 5. göze geçti. 8'den büyük, durdu.	`12 8 7 19 16 10 13 25 21 11 5 4`
10	5	13, 7	İkisinden de küçük, 2. göze geldi.	`12 8 5 19 16 10 7 25 21 11 13 4`
11	4	25, 19	İkisinden de küçük, 3. göze geldi.	`12 8 5 4 16 10 7 19 21 11 13 25`

Tur sonunda dizi şu hâli alır:

```
12  8  5  4  16  10  7  19  21  11  13  25
```

4.3 Kazanç Nerede?

Başlangıçta 10. gözde olan 5, yalnız 13 ve 7 ile karşılaştırılarak 2. göze geldi. Hoca önce "3 karşılaştırma" dedi, hemen 2 karşılaştırma olarak düzeltti. Son gözdeki 4 de yalnız 25 ve 19 ile karşılaştırılarak 2 karşılaştırmada 3. göze geldi. Insertion Sort'ta ise 4'ü başa taşımak için 11. gözden en başa kadar bütün elemanlarla tek tek karşılaştırmak gerekirdi. Avantaj budur.

4.4 $h = 1$ Turu

h her turda 3'e bölünür: $4/3 = 1$ (tam sayı bölmesi). Artık $h = 1$ olduğu için bu aşama klasik Insertion Sort'tur. Hoca ilk adımları sözlü izledi: 8, 12'den küçük olduğu için öne geçer. 5, önce 12'yi sonra 8'i geçer. 4 için bir adım daha yapılır ve 4 en başa gelir. 16 ise yerinde kalır. Hoca "bundan sonrası Insertion Sort" diyerek izlemeyi kesti. Aşağıdaki tabloda $i = 5$'ten sonraki satırlar (ara adım eklendi).

i	Eleman	Karşılaştırma sayısı	Tur sonunda dizi
1	8	1	`8 12 5 4 16 10 7 19 21 11 13 25`
2	5	2	`5 8 12 4 16 10 7 19 21 11 13 25`
3	4	3	`4 5 8 12 16 10 7 19 21 11 13 25`
4	16	1	`4 5 8 12 16 10 7 19 21 11 13 25`
5	10	3	`4 5 8 10 12 16 7 19 21 11 13 25`
6	7	5	`4 5 7 8 10 12 16 19 21 11 13 25`
7	19	1	`4 5 7 8 10 12 16 19 21 11 13 25`
8	21	1	`4 5 7 8 10 12 16 19 21 11 13 25`

i	Eleman	Karşılaştırma sayısı	Tur sonunda dizi
9	11	5	`4 5 7 8 10 11 12 16 19 21 13 25`
10	13	4	`4 5 7 8 10 11 12 13 16 19 21 25`
11	25	1	`4 5 7 8 10 11 12 13 16 19 21 25`

> **Ders sonrası not:** Derste toplamlar söylenmedi. Bu dizi için sayım şöyledir: Shell Sort, $h = 4$ turunda 12 karşılaştırma ve 9 yer değiştirme, $h = 1$ turunda 27 karşılaştırma ve 19 yer değiştirme yapar; toplam 39 karşılaştırma ve 28 yer değiştirme eder. Aynı dizide düz Insertion Sort 55 karşılaştırma ve 48 yer değiştirme yapar. Düz Insertion Sort'ta 4 gerçekten 11 karşılaştırmayla başa gelir.

4.5 Büyük Dizide Adımlar

Hoca aynı mantığı büyük bir dizi için de düşündürdü: $N = 1200$ ve adım büyüklüğü 3 olsun. İlk turda $h = 400$ olur ve 400'er 400'er ileriye bakılır. Sonra h yaklaşık 130 olur, sonra 40 küsur, ... ve en son 1. Sonlarda kalan küçük bir eleman tek seferde 400 adım öne gelir. Her seferinde adım büyüklüğü 3'te 1'e indirilerek sonlarda kalan küçükler hızla öne taşınır.

> **Ders sonrası not:** Hoca "130 falan" ve ardından "43 falan" dedi. $400/3$ tam sayı bölmesiyle 133'tür, $133/3$ ise 44'tür. Hocanın 43'ü, yaklaşık aldığı 130'un üçte biridir. Tam dizi: $400, 133, 44, 14, 4, 1$.

5. Örnek Üzerine Öğrenci Soruları

5.1 `swap` Değişkeniyle Erken Durulabilir mi?

Soru: h 1'e inene kadar devam etmek yerine, Bubble Sort'taki gibi bir `swap` değişkeni tutsak ve örneğin $h = 130$ iken hiç yer değiştirme olmadıysa kessek daha iyi olmaz mı?

Cevap: Hoca soruyu önce mantıklı buldu, sonra çekincesini söyledi. Burada dizinin içinde ayrı alt diziler vardır: Örneğimizde 21, 16, 12 bir dizidir; 13, 7, 5 başka bir dizidir. Bu alt diziler kendi içinde sıralı olabilir, ama bu bütün dizinin sıralı olduğu anlamına gelir mi? $h = 130$ iken yer değiştirme olmaması, bir adım sonra da olmayacağı anlamına gelir mi? Hoca "ezbere konuşmayayım, hiç denemedim" diyerek kesin cevap vermedi ve konuya bir sonraki derste bakılmasını istedi. Büyükten küçüğe sıralı bir dizide denenirse hemen anlaşılacağını da ekledi.

> **Ders sonrası not:** Hocanın çekincesi doğrudur, erken durmak yanlış sonuç verebilir. Karşı örnek dersin kendi dizisinden çıkar. $h = 4$ turunun sonucu olan `12 8 5 4 16 10 7 19 21 11 13 25` dizisine yeniden $h = 4$ uygulanırsa 8 karşılaştırma yapılır, hiç yer değiştirme olmaz. Ama dizi sıralı değildir. Yani bir h turunda yer değiştirme olmaması yalnız alt dizilerin sıralı olduğunu gösterir. Hocanın önerdiği büyükten küçüğe sıralı dizi ise bu soruyu çözmez: Bölüm 6.3'teki dizide $h = 4$ turunda 12, $h = 1$ turunda 17 yer değiştirme olur, yani bayrak hiç devreye girmez.

5.2 Gezinme Sırası Fark Eder mi?

Soru: Önceki derste 4'ten 8'e, 8'den 12'ye geçerek aynı alt diziye arka arkaya bakmıştık. Bu derste 4'ten 5'e geçtik. Hangisi doğru?

Cevap: İkisi de doğrudur ve birebir aynı işi yapar. Önceki anlatımda alt dizilerin kendi içinde karşılaştırıldığı daha rahat görülüyordu. Bu derste ise i birer birer artırıldı, çünkü akış diyagramı böyle daha rahat anlaşılıyor. Karmaşıklıkları da aynıdır. Hoca kimsenin aklında yanlış bir şey kalmasın diye bu sorunun sorulmasına sevindi.

> **Ders sonrası not:** İki sıra da bu dizide $h = 4$ turunda aynı sonucu, aynı 12 karşılaştırmayı ve aynı 9 yer değiştirmeyi verir. Çünkü alt diziler birbirinden bağımsızdır. Öğrencinin sözündeki 12. göz bu dizide yoktur, $N = 12$ iken son göz 11'dir. Bu alt dizinin son gözü 8'dir.

6. Shell Sort'un Akış Diyagramı

6.1 Tasarım

Hoca algoritmayı adım adım tasarladı. Adım büyüklüğü dışarıdan alınabilir, ama çok kullanılanlardan biri 3 olmasıdır. Başlangıçta $h = N/\text{stepSize}$ alınır. En son adımda $h = 1$ olacağı için klasik Insertion Sort yapılmış olur.

Kolay takip edilsin diye i, h'den başlatılıp birer birer artırılarak $N - 1$'e kadar götürülür. j de i'den başlar. İç döngünün iki şartı vardır:

- $j \geq h$**:** Bir elemanı kendisinden h önceki elemanla karşılaştırabilmek için o gözün var olması gerekir. $h = 4$ iken $j = 3$ olsa 4 öncesi yoktur. j en az 4 olabilir.
- $a[j] < a[j-h]$**:** Sıralamada $a[j]$ küçükse başa taşınması gerekir. Hoca bu şartı yazarken önce tereddüt etti, "büyük olduğu sürece" dedi, sonra ilk yazdığı "küçük olduğu sürece" hâlinin doğru olduğunu söyledi.

İki şart sağlandığı sürece $a[j]$ ile $a[j-h]$ yer değiştirir ve $j = j - h$ yapılır. Hoca önce rahat anlaşılsın diye en temel hâl olan exchange'i yazdı. Bir tur bitince ne yapılacağını sordu. Bir öğrenci "h'yi adım sayısına bölüp h'ye atamalıyız" dedi. Bu işlem $h = 1$ turu da yapılana kadar sürer. En son dizi yazdırılır.

```
Giriş: N, a[N], stepSize
h = N / stepSize
h >= 1 olduğu sürece:
    i = h'den N-1'e kadar (birer artarak):
        j = i
        (j >= h) ve (a[j] < a[j-h]) olduğu sürece:
            a[j] <-> a[j-h]
            j = j - h
    h = h / stepSize
Çıkış: a[N]
```

6.2 İç Döngünün İzlenmesi

Hoca iç döngüyü küçük bir örnekle gösterdi. $h = 4$ olsun ve $j = 5$ iken 1. gözde 5, 5. gözde 2 bulunsun. $a[5] = 2$, $a[1] = 5$'ten küçüktür. 2 ile 5 yer değiştirir, j 4 azalıp 1 olur. 1'in 4 gerisinde eleman olmadığı için döngüden çıkılır. $j = 10$ olsaydı 10. göz 6. gözle, ardından 6. göz 2. gözle karşılaştırılır ve orada kesilirdi.

> **Ders sonrası not:** Hoca $j \geq h$ şartını anlatırken "dördün dört öncesi üçtür" gibi söyledi; doğrusu 4'ün 4 öncesi 0. gözdür. $j = 3$ için 4 öncesi yoktur, hocanın sonraki cümlesi de budur.

6.3 En Kötü Durumda Hangi Karşılaştırmalar Yapılır?

Hoca karşılaştırmaların hangi sırayla yapıldığını $N = 12$, $h = 4$ için özetledi. Bunların hepsinin yapılması için iç döngünün şartının hiç bozulmaması gerekir. Dizi büyükten küçüğe sıralıysa bunların hepsi yapılmak zorundadır, yoksa bazıları yapılmaz. Hoca bunun için tahtaya büyükten küçüğe sıralı bir dizi yazdı:

```
indeks:  0   1   2   3   4   5   6   7   8   9  10  11
dizi   : 21  15  14  13   8   7   7   6   5   4   3   2
```

i	Sırayla karşılaştırılan gözler
4	4 ile 0
5	5 ile 1
6	6 ile 2
7	7 ile 3
8	8 ile 4, sonra 4 ile 0
9	9 ile 5, sonra 5 ile 1
10	10 ile 6, sonra 6 ile 2
11	11 ile 7, sonra 7 ile 3

i	Sırayla karşılaştırılan gözler

Yani en sondaki 2, önce 7. gözle, sonra 3. gözle karşılaştırılır ve orada biter. Bir sonraki turda $h = 4/3 = 1$ olur ve burası Insertion Sort gibi çalışır.

> **Ders sonrası not:** Derste toplamlar söylenmedi. Bu dizide $h = 4$ turu sonucunda `5 4 3 2 8 7 7 6 21 15 14 13` elde edilir ve 12 karşılaştırma, 12 yer değiştirme yapılır. $h = 1$ turu 25 karşılaştırma, 17 yer değiştirme yapar. Toplam 37 karşılaştırma, 29 yer değiştirmedir. Düz Insertion Sort aynı dizide 66 karşılaştırma, 65 yer değiştirme yapar.

7. Sıralama Algoritması Seçimi Üzerine Sorular

7.1 Baş Tarafın Sıralı Olması Bir Avantaj mı?

Soru: Insertion ve Selection Sort'ta her adımda baştan itibaren sıralanmış bir kısım oluşuyor. Sıralarken bu sıralı kısmı başka bir işlem için de kullanmamız gerekiyorsa bu bir avantaj sayılır mı? Shell Sort'ta baş taraf böyle oluşmuyor.

Cevap: Yapılacak işe bağlıdır. O tarafın sıralı olması kullanılıyorsa bu da bir avantajdır. Shell Sort'ta ise baş tarafın sıralı olup olmadığına bakmak gerekir, kendiliğinden bilinmez. Hoca, arkadaşlarının `swap` sorusuna da burada döndü: Alt diziler sıralı olsa da dizinin tamamı sıralı olmayabilir. "Zannetmiyorum, ama emin de değilim, bakacağım" dedi.

> **Ek bilgi (derste anlatılmadı):** İki algoritmanın sıralı kısmı aynı anlama gelmez. Selection Sort'ta baştaki k eleman dizinin en küçük k elemanıdır ve son yerlerindedir. Insertion Sort'ta ise baştaki k eleman yalnız dizinin ilk k elemanının sıralı hâlidir, sonraki elemanlar bunların arasına girebilir. Dersin dizisinde $i = 3$ turu sonunda baş taraf `8 13 19 21` olur, oysa dizinin en küçüğü 4'tür.

Hoca bu tür durumlara bakış açısını Quick Sort ile örnekledi. Quick Sort normalde $n \log n$ karmaşıklıkla en iyi sıralama algoritmasıdır, ama size sıralı bir dizi verilirse n^2 karmaşıklığa kadar düşer. Yine de avantajlıdır, çünkü dizinin sıralı gelmesi zaten çok beklenen bir şey değildir. Hocanın deyişiyle bu "yüzyılda bir" karşınıza çıkacak bir durumdur. Olasılık olarak dizinin sırasız olması beklenir.

> **Ders sonrası not:** Quick Sort'un sıralı dizide n^2'ye düşmesi, pivotun ilk veya son eleman seçildiği sürüm içindir. Bu sürümde her bölmede bir taraf boş kalır ve $n(n-1)/2$ karşılaştırma yapılır. Pivot rastgele ya da üçün ortancasıyla (median-of-three) seçilirse sıralı dizi bu kötü durumu doğurmaz.

7.2 En Avantajlı Algoritmayı Bulan Bir Algoritma Var mı?

Soru: Sıralama algoritmaları arasında duruma göre çok fark olabiliyor. Hangi algoritmanın daha avantajlı olduğunu bulan bir algoritma var mı?

Cevap: Hoca soruyu çok güzel buldu. Bunun için algoritmalar çalıştırılıp süreleri ölçülür. Çünkü durumlar çok değişir: Dizi birazcık sıralıysa ne olur, küçükten büyüğe sıralıysa ne olur, büyükten küçüğe sıralıysa ne olur? Bazı farklar gözle anlaşılabilir, bazıları boyut arttıkça ortaya çıkar. Hoca birkaç örnek verdi:

- **Bubble ve Selection:** En beğenilmeyen Bubble Sort bile, dizi sıralıysa Selection Sort'un önüne geçer.
- **Merge Sort:** $n \log n$ karmaşıklıklı algoritmaların dönem sonuna doğru görüleceğini hatırlatan hoca, Merge Sort'un bunların içinde en çok kullanılanlardan biri olduğunu söyledi. Ama Merge Sort yedek dizi kullanır. Çok eleman varsa ve yedek dizi için yer yoksa kullanılmaması gerekir.
- **Java ve Quick Sort:** Java Quick Sort kullanır. Quick Sort özyinelemelidir (recursive), kendini çağırır, diziyi küçük parçalara böler ve her parçayı ayrı sıralar. Küçük parçalara inildiğinde Insertion Sort'a geçer. Baştan küçük bir dizi verilirse de hiç Quick Sort yapmadan doğrudan Insertion Sort kullanır. Sebebi, küçük dizi parçalarında Insertion Sort'un

daha hızlı olmasıdır, çünkü Quick Sort'un sürekli fonksiyon çağırma ek yükü (overhead) vardır.

Ders sonrası not: Java için bu ifade geneldir. `int[]` gibi ilkel tiplerde `Arrays.sort` büyük olasılıkla çift pivotlu Quick Sort kullanır ve küçük parçalarda Insertion Sort'a geçer. Nesne dizilerinde ise büyük olasılıkla kararlı bir Merge Sort türevi (TimSort) kullanılır.

Hocanın vurgusu: Tek bir doğru yoktur. Her algoritmanın avantajları ve dezavantajları bilinmeli, kullanırken bunlar göz önünde bulundurulmalıdır. Doğrusu da budur.

Hoca bu tür ayrıntıları özellikle sevdiğini söyledi. Bilgisayar ortamında yazılan algoritmaların zamanı ve yeri iyi kullanması gittikçe daha önemli hâle geliyor ve bilgisayar mühendisinin verimli çözümler sunması gerekiyor. Bunları akılda tutmak gerekmese bile, gerektiğinde bir yerlere bakarak daha iyi seçim yapabileceğini bilmek önemlidir.

8. Exchange Yerine Kaydırma: `tmp` ile İyileştirme

Hoca akış diyagramında anlaşılır olsun diye exchange kullandığını, sürekli yer değiştirmek yerine Insertion Sort'taki gibi kaydırma da yapılabileceğini gösterdi:

- Döngüden önce `tmp = a[j]` alınır.
- İç döngü şartı `tmp < a[j-h]` olur.
- Döngü içinde yalnız `a[j] = a[j-h]` yazılır.
- Döngüden çıkınca `a[j] = tmp` yazılır.

Böylece exchange engellenmiş ve algoritma biraz daha iyileştirilmiş olur.

Ek bilgi (derste anlatılmadı): Derste C kodu yazılmadı. Aşağıdaki program, akış diyagramının iki hâlinin bu not için yazılmış karşılığıdır. `gcc -Wall` ile uyarısız derlenip çalıştırıldı. İki fonksiyon da dersin dizisinde aynı sonucu verir. Çıktı kodun altındadır.

```c
#include <stdio.h>

void shell_sort_exchange(int a[], int n, int step_size) {
    int h = n / step_size, i, j, gecici;
    while (h >= 1) {
        for (i = h; i <= n - 1; i++) {
            j = i;
            while (j >= h && a[j] < a[j - h]) {
                gecici = a[j];
                a[j] = a[j - h];
                a[j - h] = gecici;
                j = j - h;
            }
        }
        h = h / step_size;
    }
}

void shell_sort_tmp(int a[], int n, int step_size) {
    int h = n / step_size, i, j, tmp;
    while (h >= 1) {
        for (i = h; i <= n - 1; i++) {
            j = i;
            tmp = a[j];
            while (j >= h && tmp < a[j - h]) {
                a[j] = a[j - h];
                j = j - h;
            }
            a[j] = tmp;
```

```
        }
        h = h / step_size;
    }
}

void yaz(const int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main(void) {
    int a[12] = {21, 8, 13, 19, 16, 11, 7, 25, 12, 10, 5, 4};
    int b[12] = {21, 8, 13, 19, 16, 11, 7, 25, 12, 10, 5, 4};
    shell_sort_exchange(a, 12, 3);
    yaz(a, 12);
    shell_sort_tmp(b, 12, 3);
    yaz(b, 12);
    return 0;
}
```

```
4 5 7 8 10 11 12 13 16 19 21 25
4 5 7 8 10 11 12 13 16 19 21 25
```

9. Shell Sort Ne Zaman Kullanılmalı?

Birincisi, Shell Sort daha büyük diziler için iyidir. 12 elemanlı bir diziyi "parçalara böleyim, dörder dörder atlasın" diye uğraşmak gerçekte çok gerekli değildir.

> **Hocanın vurgusu:** Bu ayrımı yapmanız çok çok önemli. $n = 10$ iken zaten bir şey olmaz, n yüz milyon iken olur. 12 elemanınız varken hangisini kullandığınız çok önemli değildir.

Hoca karmaşıklığı $n^{3/2}$ de olsa Shell Sort'u sonuçta n^2'ye yakın bir algoritma olarak gördüğünü söyledi. Uygun olduğu yer 100, 200, 300 elemanlı gibi daha büyük dizilerdir. 1 milyon elemanınız varsa Quick Sort gibi algoritmalar kullanılmalı, bunlara hiç bakılmamalıdır.

> **Ders sonrası not:** "Yakın" sözü mertebeler için dikkatli okunmalıdır. $n^2/n^{3/2} = \sqrt{n}$ olduğu için aradaki oran n ile büyür: $n = 10$ için yaklaşık 3,2; $n = 100$ için 10; $n = 10^6$ için 1000; $n = 10^8$ için 10000 kattır. Hocanın Bölüm 3.1'deki "1 milyon, 2 milyon oldukça fark oluşur" sözü de budur. Her iki mertebe de $n \log n$'den kötüdür; hocanın milyonluk dizilerde Quick Sort önermesinin sebebi budur.

İkincisi, Shell Sort da Insertion Sort gibi tamamen rastgele sayılardan değil, biraz sıralı sayılardan oluşan dizilerde avantajlıdır. Zamanında sıralı olup, üzerindeki hareketler ve gelen giden elemanlar yüzünden sırası hafifçe bozulmuş ama çok da sırasız olmayan diziler buna örnektir.

Üçüncüsü, genel olarak Insertion Sort'tan da Selection Sort'tan da daha hızlı bir algoritmadır.

10. Çalışma Süresini Deneysel Olarak Ölçmek

Hoca, burada anlatılanları görmek için kendi başımıza denemeler yapmamızı önerdi. Yarı sıralı dizilerde programı çalıştırıp çalışma zamanını ölçeriz. Ölçüm şöyle yapılır:

- Ekrana hiç yazdırma yapılmaz, yani giriş çıkış (I/O) işlemi olmaz.
- İşlemlerin başladığı yerin üzerine, sistem zamanını ölçen bir fonksiyon konur.
- Aynı fonksiyon işlemlerin bittiği yere de konur. Böylece zaman ölçülür.

Hoca bunun için büyük diziler kullanılmasını istedi. Elle girmek yerine diziyi dosyadan okutabilir ya da programa rastgele sayı ürettirebilirsiniz. Önerdiği denemeler şunlardır:

- 1000 ya da 5000 elemanlık bir dizide Selection Sort ve Insertion Sort'u deneyin.
- Dizinin tam yarısına kadar sıraladıktan sonra yeniden deneyin. Hangisi daha hızlı?
- Büyükten küçüğe sıralı dizi için deneyin.

  Ek bilgi (derste anlatılmadı): C'de bu iş için `time.h` içindeki `clock()` kullanılabilir. İşlemden önce ve sonra alınan iki değerin farkı `CLOCKS_PER_SEC` değerine bölünerek saniye elde edilir. Hocanın altyazıda anlaşılmayan fonksiyonu bu olmayabilir.

Hocaya göre bunlar eğlenceli şeylerdir. Asıl kazanç da şudur: Verimli yazmaya alışan kişi, ileride daha büyük ve gerçek hayatta kullanılan problemlerde farkına bile varmadan daha verimli kod yazar.

11. Akış ve Kod Üzerine Son Sorular

Hoca, Shell Sort'un genelde çok konuşulmayan bir algoritma olduğunu, bu derste ise onu "yıldız yaptığını" söyleyerek şakalaştı. Ardından iki soru daha geldi.

11.1 İki `for` ile Yazılan Kod

Soru: Önceki derste anlatılan akışa göre kodladım. Siz iki `while` ve bir `for` kullandınız, bende bir `for` daha var. Adım büyüklüğü 3 iken siz 3-0'a, sonra 4-1'e bakıyorsunuz. Ben ise önce 3-0, sonra 6-3-0, sonra 9-6-3-0 diye gidiyorum. 4'e geçmek için indeksi artırmam gerekiyor, bunu da 3 ile 6 arasında kalacak ayrı bir `for` ile yapıyorum.

Cevap: Bu da doğrudur. Hoca kendi anlatım sırasını değiştirmesinin sebebini yineledi: Algoritmayı anlatırken aynı alt diziye arka arkaya bakmak daha rahat anlaşılır, kod yazarken ve akış çizerken ise i'yi birer artırmak daha anlaşılırdır. İkisi de aynı işi yapar. Hoca, kodun daha iyileştirilip iyileştirilemeyeceğine ayrıca bakacağını söyledi.

11.2 Akış Diyagramında Kısaltma Kullanılabilir mi?

Soru: Bu akışta bir işlemi kısaca tek kutuyla gösterdiniz, diziyi de tek giriş kutusunda aldınız. Bazı hocalar buna çok dikkat ediyor. Bizim böyle yazmamızda sorun olur mu?

Cevap: Hoca birinci sınıf ile bugünü ayırdı. Birinci sınıf öğrencisi örneğin $a[n]$ elemanını yazdırdığını zanneder, oysa dizi 0'dan başlar ve böyle bir eleman yoktur. Bunu oturtmak zaman alır. Bu yüzden hoca dizilerin döngüyle nasıl okunup yazıldığını bütün algoritmalarında açık açık gösterirdi, dönemin ortasından sonra da kısaltmaya geçerdi. Değiş tokuş işlemi için de aynısı geçerlidir: Birinci sınıf öğrencisine geçici bir yer olduğunu ve değerlerin nasıl alındığını tek tek söylemek gerekir. İkinci sınıf ikinci dönemde bu nokta çok geçilmiştir ve kısaltmaların bir sakıncası yoktur. Hoca, diğer hocaların nasıl yaptığını bilmediğini de belirtti. Ölçü, okuyanın "burada ne demek istedi?" demeyeceği açıklıktır.

Dersin Toparlaması

Bu derste önce üç n^2 algoritma karşılaştırıldı. Bubble Sort, her kontrolden sonra yer değiştirme ihtiyacı yüzünden geride kalır. Insertion Sort ise yalnız sırası gelen elemanın yerini aradığı ve yerini bulunca durduğu için öne çıkar, en çok da yarı sıralı dizilerde. Shell Sort, Insertion Sort'un sonlarda kalan küçük elemanları yavaş taşıma sorununu zıplayarak çözer. 12 elemanlı örnekte $h = 4$ ve $h = 1$ turları izlendi, ardından akış diyagramı ve `tmp` ile iyileştirilmiş hâli kuruldu.

Asıl mesaj seçimle ilgilidir: Tek doğru algoritma yoktur. Fark n büyüdükçe ortaya çıkar ve dizinin yapısına, yer kısıtına bağlıdır. Bu yüzden algoritmaları farklı boyut ve durumlarda çalıştırıp süre ölçmek önerildi. `swap` değişkeniyle erken durma sorusu sonraki derste konuşulmak üzere bırakıldı, $n \log n$ algoritmalar da dönem sonuna doğru görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.